

Applied Deep Learning

Chapter 4: Convolutional Neural Networks

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Idea of Pooling: Smoothing Filters

The output of convolution can be **jittering** which can come from **spatial correlation**: **Pooling** acts as a filter by sliding over the **extracted features** and pooling out a function of each subpart

- Pooling can **reduce** the **jittering** behavior
- It can mix **extracted features** and potentially improve **shift-invariance**

Shift-Invariance

shift-invariance refers to robustness against simple geometric transform of input

- + How do we do the pooling?
- There are several pooling techniques but popular ones are **max-** and **mean-pooling**

Max-Pooling

For max-pooling, we use a filter of size $L \times L$ and slide over the **feature map**

$$\mathbf{Y} = \begin{bmatrix} Y_{1,1} & Y_{1,2} & \cdots & Y_{1,M} \\ Y_{2,1} & Y_{2,2} & \cdots & Y_{2,M} \\ \vdots & \vdots & \vdots & \vdots \\ Y_{N,1} & Y_{N,2} & \cdots & Y_{N,M} \end{bmatrix}$$

$\max \{ \mathbf{Y}_{1,1} \mathbf{Y}_{1,2} \mathbf{Y}_{i,j} \}$

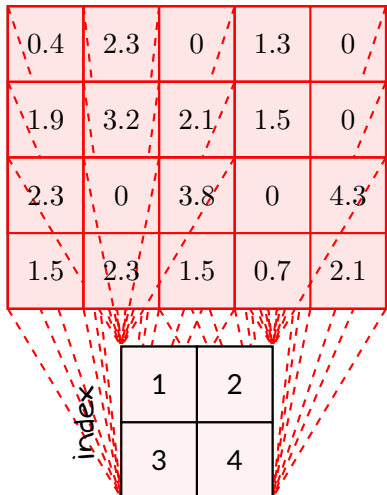
In each window, we pool the **maximum**

$$\hat{\mathbf{Y}} = \begin{bmatrix} \hat{Y}_{1,1} & \hat{Y}_{1,2} & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

Attention

In **max-pooling**, we also track **index of maximizer**: we need it in **backpropagation**

Max Pooling: Numerical Example



3.2	3.2	2.1	1.5
3.2	3.8	3.8	4.3
2.3	3.8	3.8	4.3

4	3	3	3
2	4	3	4
1	2	1	2

index

Mean-Pooling

In mean-pooling, we again use an $L \times L$ filter and slide over the **feature map**

$$\mathbf{Y} = \begin{bmatrix} Y_{1,1} & Y_{1,2} & \dots & Y_{1,M} \\ Y_{2,1} & Y_{2,2} & \dots & Y_{2,M} \\ \vdots & \vdots & \vdots & \vdots \\ Y_{N,1} & Y_{N,2} & \dots & Y_{N,M} \end{bmatrix}$$

A 2×2 window is highlighted in blue, containing elements $Y_{1,1}$, $Y_{1,2}$, $Y_{2,1}$, and $Y_{2,2}$. A dashed blue arrow points from this window to the expression $\text{mean} \{ \mathbf{Y}_{1,1} \mathbf{Y}_{1,2} \mathbf{Y}_{i,j} \}$.

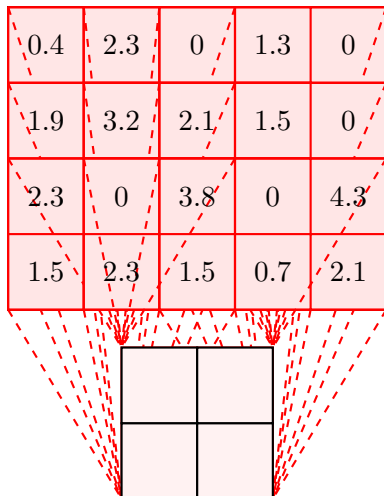
In each window, we pool the **average**

$$\hat{\mathbf{Y}} = \begin{bmatrix} \hat{Y}_{1,1} & \hat{Y}_{1,2} & \dots & \dots \\ \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \end{bmatrix}$$

Two arrows point from the 2×2 window in the matrix $\mathbf{Y}$ to the first two elements of the first row in the matrix $\hat{\mathbf{Y}}$, illustrating the pooling operation.

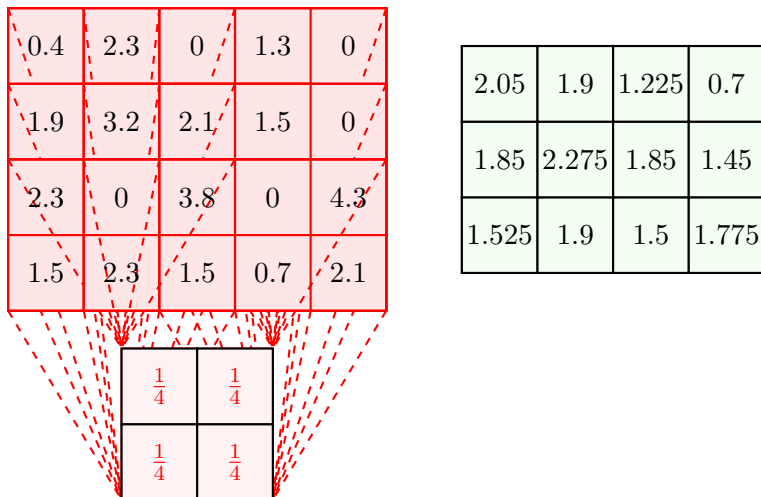
We can look at mean-pooling as a **convolution** with **uniform kernel**

Mean-Pooling: Numerical Example



2.05	1.9	1.225	0.7
1.85	2.275	1.85	1.45
1.525	1.9	1.5	1.775

Mean-Pooling: Numerical Example



mean-pooling $\equiv$ convolution with uniform kernel

Advanced Pooling: Use a General Function

We could in general replace **max or mean operator** with a general function

$$\mathbf{Y} = \begin{bmatrix} Y_{1,1} & Y_{1,2} & \dots & \dots & Y_{1,M} \\ Y_{2,1} & Y_{2,2} & \dots & \dots & Y_{2,M} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ Y_{N,1} & Y_{N,2} & \dots & \dots & Y_{N,M} \end{bmatrix} \xrightarrow{\quad} \Pi(\cdot) : \mathbb{R}^{L \times L} \mapsto \mathbb{R}$$

Note: In the original image, the first two columns of the matrix Y are highlighted with a blue box, and the labels Y_{1,1}, Y_{1,2}, Y_{2,1}, Y_{2,2} are in blue. A dashed blue arrow points from this box to the function symbol \Pi(\cdot). The rest of the matrix elements are in red.

Typical functions are **norms**, e.g., ℓ_2 -norm

$$\hat{\mathbf{Y}} = \begin{bmatrix} \hat{Y}_{1,1} & \hat{Y}_{1,2} & \dots & \dots \\ \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \end{bmatrix}$$

Note: In the original image, the matrix \hat{Y} is entirely green. Two black arrows point from the first two columns of the input matrix Y to the first two columns of the output matrix \hat{Y}.

We can even replace it with a **small NN**!

Pooling with Stride

Similar to convolution, we can apply **pooling** with **stride**

- We usually do not use **fractional stride** with **pooling**
 - ↳ Upsampling the input, only adds zeros
 - ↳ Extra zeros usually do not make any gain: *think of max-pooling for instance*
- Integer **stride** can be seen as downsampling
 - ↳ We apply pooling with **unit stride**
 - ↳ We down-sample output with **sampling factor = stride**
- In practice, we often leave integer **strides** for **pooling** layer
 - ↳ We apply convolution with **unit stride**
 - ↳ If we need to down-sample, we do it later in the **pooling** layer

Pooling: Few Remarks

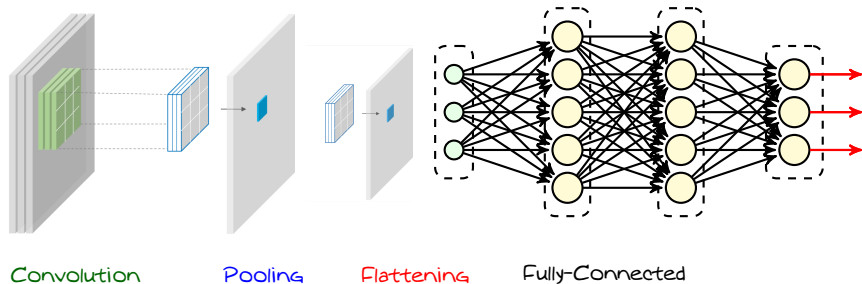
- + *Do we always pool after convolution?*
- Not really! In many architectures pooling is applied every couple of convolutional layers

It is worth noting that

- In many architectures **pooling** is applied **every couple of convolutional layers**
 - ↳ We apply **multiple** convolutional layers
 - ↳ We then apply a **pooling** layer **with stride**
- Most **poolings** have no **learnable parameters**; thus, **they are cheap**
 - ↳ **Max** and **mean-pooling** have **no** weights
- **Filter size** for **pooling** can be different from the **convolutional layers**
 - ↳ They are typically in **the same range**

Output FNN: *Flattening*

Let's recall our simple CNN

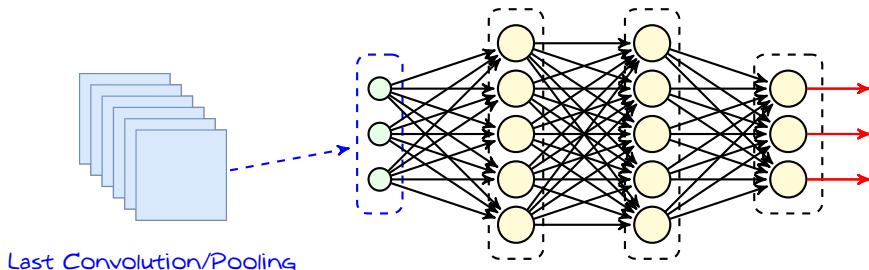


Once we are over with convolution and pooling we *flatten* final *feature tensor*

Flattening

In *flattening*, we sort all entries of the *feature tensor* into a vector

Flattening



Say we have a *feature tensor* with K channels

↳ each channel is an $N \times M$ map after last *convolution* or *pooling*

We then have an input to the FNN with NMK entries

After *flattening* everything goes as before through the *fully-connected FNN*